React + TypeScript · 10-Day Training Plan

Day 3 — React Fundamentals

Your simple, friendly guide for today

COMING FROM	KnockoutJS / .NET (C#) → moving to React + TypeScript
TODAY'S GOAL	Build a screen from small components. You describe the UI for your data; React puts it on the page and updates it for you.

How to use this: Read **Section 1 (Learning Plan)** for each idea with a small example. Then read **Section 2 (Detailed Notes)** slowly, top to bottom — one read should make it click. Keep **Section 3 (Common Mistakes)** handy while you practise, and copy **Section 4 (Concept Notes)** by pen for recall & interviews.

1 Learning Plan

Welcome to React. Today is about the **building blocks** — the small pieces you snap together to make a screen. You already know JavaScript (Day 1) and TypeScript (Day 2). React just gives you a nice way to turn data into UI (the stuff on screen).

Think of it like this: in KnockoutJS you wrote a viewmodel and an HTML template, and KO wired them together. In React, **one file holds both** — the logic and the markup live side by side in a function. That function is called a *component*.

Today's goal is to build three small pieces and stack them: **EmployeeCard** (shows one person) -> **DepartmentCard** (a box that holds cards) -> **Dashboard** (the whole screen).

We will NOT cover state, events, or hooks today. Those come tomorrow (Day 4). Today is just: how do I show data on screen with React.

Part A — The Core Fundamentals

1. JSX — HTML-looking code inside JavaScript

JSX is a special syntax that lets you write tags like `<h2>` right inside your `.tsx` file. It is **not** a string and it is **not** real HTML — it is JavaScript in disguise. Anything inside curly braces `{ }` is live JavaScript that gets evaluated. So `{name}` drops the value of the `name` variable into the page.

```
const name = "Asha";
const greeting = <h2>Hello, {name}!</h2>;
```

Coming from KnockoutJS / C#: like Razor's `@Model.Name` in a `.cshtml` file — code and markup mixed together, but here it is JavaScript, not C#.

You'll be able to: write markup and drop live values into it without string concatenation.

---

2. Function Components — your reusable UI pieces

A component is just a **function that returns JSX**. Its name must start with a **Capital letter** (React uses that to tell your components apart from plain HTML tags like `<div>`). You use a component by writing it like a tag: `<Hello />`.

```
function Hello() {
  return <p>Hello from a component!</p>;
}
```

Coming from KnockoutJS / C#: like a KO component or a .NET partial view / user control — a named, reusable chunk of UI.

You'll be able to: make your own tags (like `<EmployeeCard />`) and reuse them anywhere.

---

3. Props — passing data INTO a component

Props (short for "properties") are the **inputs** to a component. You pass them like HTML attributes: `<EmployeeCard firstName="Asha" />`. Inside the component they arrive as one object. We type that object with an **interface** (this is your Day 2 knowledge paying off) so TypeScript checks you passed the right things. Props are **read-only** — a component never changes its own props.

```
interface EmployeeCardProps {
  firstName: string;
  lastName: string;
  department: string;
}

function EmployeeCard({ firstName, lastName, department }: EmployeeCardProps) {
  return (
    <div className="employee-card">
      <h3>{firstName} {lastName}</h3>
      <p>{department}</p>
    </div>
  );
}
```

Coming from KnockoutJS / C#: like passing parameters into a C# method, or `params` into a KO component — but they flow **one way, in only**.

You'll be able to: build one `EmployeeCard` and reuse it for every employee, just by changing the props.

---

4. Component Composition + `children` — putting components inside components

Composition just means **nesting** components to build bigger ones. A component can also accept whatever you put *between* its opening and closing tags — that arrives as a special prop called `children`. We type `children` as `ReactNode` (React's word for "anything you can render": text, tags, other components). This is how you build a reusable "box" like a `DepartmentCard` that wraps any content.

```tsx
import type { ReactNode } from "react";

interface DepartmentCardProps {
  name: string;
  children: ReactNode;
}

function DepartmentCard({ name, children }: DepartmentCardProps) {
  return (
    <section className="department-card">
      <h2>{name}</h2>
      {children}
    </section>
  );
}
```

Coming from KnockoutJS / C#: `children` is like KO's `<!-- ko template -->` slot or a Razor layout's `@RenderBody()` — a hole where outside content gets dropped in.

You'll be able to: wrap several `EmployeeCard`s inside one `DepartmentCard` box.

---

5. Project Structure — where files live

A React + TypeScript project is just folders of files. Each component usually gets its **own file** in a `components/` folder, named after the component (`EmployeeCard.tsx`). You then `import` it wherever you need it. Good structure = easy to find things later.

```tsx
import { EmployeeCard } from "./components/EmployeeCard";
```

Coming from KnockoutJS / C#: like one class per `.cs` file with namespaces and `using` statements — one component per file, `import` instead of `using`.

You'll be able to: keep each component in its own tidy file and import it by name.

(Full recommended folder layout is in Part B below.)

---

6. How Rendering Works — JSX becomes UI

Rendering means React reads your JSX and puts matching elements on the actual screen (the DOM — the browser's live page). Your app starts at a single **root** where React "mounts" (attaches) your top component. Data flows **one way**: from parent down to child through props — never back up on its own. When a component's data changes, React **re-renders** it: it runs the function again, sees the new JSX, and updates only the parts that changed. (What triggers a change is *state* — that is Day 4.)

```
import { createRoot } from "react-dom/client";
import { App } from "./App";

createRoot(document.getElementById("root")!).render(<App />);
```

Coming from KnockoutJS / C#: KO used two-way bindings and `ko.applyBindings` to sync the DOM for you. React is **one-way** and you **never touch the DOM by hand** — no `$("#id").text( ... )`, no `document.querySelector`.

You'll be able to: understand that you describe *what* the UI should look like, and React handles *how* to draw it.

---

7. Conditional Rendering — show something only sometimes

Because JSX is just JavaScript, you decide what to show using normal JS: a ternary `cond ? a : b` for either/or, or `cond && <thing />` to show a thing only when `cond` is true. This replaces "if" logic in your markup.

```
interface StatusProps {
  isActive: boolean;
}

function Status({ isActive }: StatusProps) {
  return <span>{isActive ? "Active" : "Inactive"}</span>;
}
```

Coming from KnockoutJS / C#: replaces KO's `data-bind="if: isActive"` / `visible:` and Razor's `@if ( ... )` — but it is plain JavaScript.

You'll be able to: show an "Active" / "Inactive" badge on an employee based on a boolean.

---

8. Lists & Keys — showing many items

To render a list, use JavaScript's `.map()` to turn an **array of data** into an **array of JSX**. React needs each item in a list to have a `key` — a stable, unique value (like a database `id`) so React can tell items apart when the list changes. Use the real `id`; do **not** use the array index if the list can reorder or change.

```
const departments = ["Engineering", "Sales", "HR"];

function DepartmentList() {
  return (
    <ul>
      {departments.map((dept) => (
        <li key={dept}>{dept}</li>
      ))}
    </ul>
  );
}
```

Coming from KnockoutJS / C#: replaces KO's `data-bind="foreach: items"` and Razor's `@foreach` — the `key` is React's way of tracking items, similar to a primary key.

You'll be able to: render a whole list of employees from an array with one `.map()`.

Putting It All Together — the Dashboard

Now stack the three pieces. `Dashboard` maps over employee data into many `EmployeeCard`s, and drops them (as `children`) inside a `DepartmentCard`. This is composition + props + lists + keys, all at once — and it is basically a whole small screen.

```
interface Employee {
  id: number;
  firstName: string;
  lastName: string;
  department: string;
}

const employees: Employee[] = [
  { id: 1, firstName: "Asha", lastName: "Rao", department: "Engineering" },
  { id: 2, firstName: "Ben", lastName: "Cole", department: "Engineering" },
];

function Dashboard() {
  return (
    <DepartmentCard name="Engineering">
      {employees.map((emp) => (
        <EmployeeCard
          key={emp.id}
          firstName={emp.firstName}
          lastName={emp.lastName}
          department={emp.department}
        />
      ))}
    </DepartmentCard>
  );
}
```

That `Employee` type is the same shape you built on Day 1 (trimmed down here to keep the focus on React).

Part B — Extra Topics (setup & tooling)

These are the practical bits that surround the React code. Read once, then you have your environment ready.

B1. Create a React + TypeScript project with Vite

Vite (say "veet") is the modern tool that creates the project and runs a fast dev server (a local web server that auto-reloads when you save). Run these in your terminal:

```
npm create vite@latest employee-dashboard -- --template react-ts
cd employee-dashboard
npm install
npm run dev
```

- `--template react-ts` = React **with TypeScript** already set up (`.tsx` files, `tsconfig.json`, strict mode).
- `npm run dev` starts the dev server and prints a local URL (usually `http://localhost:5173`). Open it in the browser. Save a file and the page refreshes itself.

Coming from .NET: Vite is like `dotnet new` + `dotnet watch run` combined — it scaffolds the project and hot-reloads as you code.

B2. A sensible folder structure

Vite gives you a `src/` folder. A clean starting layout:

```
src/
  components/        ← your reusable UI pieces
    EmployeeCard.tsx
    DepartmentCard.tsx
    Dashboard.tsx
  types/             ← shared TypeScript types (e.g. Employee)
    employee.ts
  App.tsx            ← top component, composes everything
  main.tsx           ← entry point: mounts <App /> to the DOM
  index.css
```

Rule of thumb: **one component per file**, file name matches the component name. Shared types (like `Employee`) live in `types/` so every component can import the same shape.

B3. JSX rules cheat sheet

Small rules that trip up everyone at first:

- **One root element.** A component must return a single top element. Wrap siblings in a fragment `<> ... </>` (an invisible wrapper that adds no extra tag).
- **`className`, not `class`.** (`class` is a reserved word in JavaScript.) Also `htmlFor` instead of `for` on labels.
- **Close every tag.** Even self-closing ones: `<img />`, `<br />`, `<input />`.
- **`{expression}` embeds JavaScript.** Anything in `{ }` is live JS. Comments inside JSX look like `{/* like this */}`.
- **camelCase props/events.** `onClick`, `tabIndex`, `strokeWidth` — not `onclick` or `tabindex`.

```
function TwoLines() {
  return (
    <>
      <p>Line one</p>
      <p>Line two</p>
    </>
  );
}
```

B4. React DevTools

Install the **React Developer Tools** browser extension (Chrome/Edge/Firefox). It adds a "Components" tab to your browser's dev tools. There you can click any component on the page and **see its props** and how components are nested. Super useful to check "did the right data reach this card?"

Coming from KnockoutJS: like inspecting your viewmodel's live values, but visual — you see the whole component tree and each component's props.

B5. ESLint + Prettier

- **ESLint** = catches likely mistakes and enforces good React/TS practices (red squiggles). The `react-ts` Vite template already includes an ESLint setup.
- **Prettier** = auto-formats your code (spacing, quotes, semicolons) so you never argue about style. Add it and enable "Format On Save" in VS Code.

You already have a `.prettierrc` in this repo, so formatting is set — just save and it tidies itself.

B6. KnockoutJS -> React mindset shift

The single biggest mental change:

KnockoutJS (old habit)	React (new way)
Viewmodel + separate HTML template	One component = logic **and** markup together
`ko.observable`, two-way binding	**One-way** data flow: props go parent -> child only
KO updates the DOM via bindings	You **describe** the UI; React updates the DOM for you
`data-bind="foreach / if / visible"`	Plain JavaScript: `.map()`, `? :`, `&&`
Manual DOM edits (`$el.text( ... )`)	**Never** touch the DOM by hand

The one-sentence version: In React you write UI as a **function of your data** — say what the screen should look like for the current data, and React makes the screen match. State (what makes data change) is tomorrow, Day 4.

Recap: JSX (markup in JS) -> function components (reusable pieces) -> props (inputs, typed with an interface) -> composition + `children` (nesting) -> rendering (one-way, React draws the DOM) -> conditional rendering (`? :` , `&&`) -> lists + keys (`.map()` with a unique `key`). You now have everything to read and build the `EmployeeCard` -> `DepartmentCard` -> `Dashboard` screen.

2 Detailed Notes — Read Once, Understand Everything

The big idea first

Here is the one sentence that makes React click:

In React, you describe what the screen should look like for the data you have right now — and React puts it on the screen for you.

That word "describe" is the whole game. You don't grab elements and update them by hand. You write a function that says "given this data, the UI looks like *this*." When the data changes later, React re-runs your description and updates the screen to match.

Compare this to KnockoutJS. In Knockout you make `observable`s and then wire up bindings in HTML with `data-bind`. You are manually connecting each piece of data to each piece of UI, and you keep those wires in your head. React removes the wiring. You just describe the result. This style has a name: **declarative** (you say *what* you want, not *how* to update it step by step). Knockout's `data-bind` is declarative for one value at a time; React is declarative for the whole component at once.

Coming from C#, think of a component like a method that returns markup. You pass in inputs (props), it returns UI. Give it different inputs, you get different UI. Nice and predictable.

Let's build up all of React fundamentals from here. We'll reuse the **Employee** idea from Day 1. Its shape:

```
// employee.ts — the Day 1 model, now typed (Day 2 skills)
export interface Employee {
  id: number;
  firstName: string;
  lastName: string;
  department: string;
  salary: number;
  isActive: boolean;
  joinDate: string;
}
```

What React is (and what a SPA is)

React is a JavaScript library for building user interfaces out of small reusable pieces called components. A component is a chunk of UI — a card, a button, a whole dashboard. You build big screens by snapping small components together, like LEGO bricks.

React is usually used to build a **SPA — Single Page Application.** "Single page" means: the browser loads *one* HTML file once, and from then on JavaScript swaps the content in and out. There are no full page reloads when you click around.

If you're picturing classic ASP.NET MVC or Razor Pages: there, each navigation is a round-trip to the server, and the server sends back a fresh HTML page. A SPA is different — it loads once, then feels more like a desktop app running inside the browser tab. The page updates in place.

JSX — it looks like HTML, but it isn't

Look at this:

```
const greeting = <h1 className="title">Hello team</h1>;
```

That HTML-looking thing inside JavaScript is called **JSX**. It is *not* HTML, and it is *not* a string. It's a special syntax React uses to describe UI right inside your code.

Here's the important part: **JSX compiles down to plain function calls.** A build tool (Vite, in your case) rewrites it before it ever runs:

```
// You write this JSX:
const badge = <span className="badge">Active</span>;

// The build tool turns it into a plain function call, roughly:
// const badge = jsx("span", { className: "badge", children: "Active" });
```

So JSX is just a friendly shortcut for calling functions that build UI objects. If you know Razor, it's a similar idea: Razor markup also compiles down to C# code that writes HTML. You write the pretty version; the tool makes the real version.

A few **rules** for JSX. Learn these five and you'll avoid 90% of beginner errors:

1. **One root element.** A component must return a single top-level element. If you need siblings with no wrapper, wrap them in a **Fragment**: `<> … </>` (an empty tag that groups things without adding a real DOM node).
2. `className` **, not** `class` **.** `class` is a reserved word in JavaScript, so JSX uses `className`. (Same reason `for` on a label becomes `htmlFor`.)
3. `{ }` **drops you back into JavaScript.** Anything inside curly braces is a JS *expression* — a value. `{firstName}`, `{2 + 2}`, `{emp.salary * 12}`. Note: expressions only, not statements. You can't put an `if` block inside `{ }`, but you can put a `? :`.
4. **Self-close empty tags.** `<img />`, `<br />`, `<input />`. The slash is required in JSX.
5. **camelCase for most attributes.** `onClick`, `tabIndex`, `maxLength`.

```
function Example() {
  const name = "Priya";
  return (
    <> {/* Fragment: groups two elements, adds no extra div */}
      <h1 className="title">Hello {name}</h1>
      <input placeholder="Search employees" />
    </>
  );
}
```

Function components — a component is just a function

This is the whole definition, and it's beautifully small:

A component is a function that returns JSX.

```
function Welcome() {
  return <h1>Welcome to the dashboard</h1>;
}
```

That's a real, complete component. Two rules:

- The name **must start with a capital letter** (`Welcome` , not `welcome`). React uses the capital letter to tell your components apart from regular HTML tags like `<div>` .
- It **returns JSX** (or `null` for "render nothing").

You use it by writing it like a tag: `<Welcome />` . React calls your function and drops the returned JSX where you placed the tag.

You may have heard of **class components** (`class X extends React.Component`). That was the old style. We use function components only — that's the modern way, and it's all you need. That's the one line I'll say about classes.

Props — how a parent passes data down

A component with hard-coded text isn't reusable. To make it reusable, the parent passes in data. That data is called **props** (short for "properties").

Think of props exactly like **method parameters in C#.** The parent calls the component and hands it arguments. Inside, you read them.

We type props with an **interface** (reinforcing Day 2). This tells you and the compiler exactly what data this component expects:

```
interface EmployeeCardProps {
  firstName: string;
  lastName: string;
  department: string;
}

function EmployeeCard(props: EmployeeCardProps) {
  return (
    <div className="card">
      <h2>{props.firstName} {props.lastName}</h2>
      <p>{props.department}</p>
    </div>
  );
}
```

The parent passes props like HTML attributes:

```
function Team() {
  return (
    <div>
      <EmployeeCard firstName="Aisha" lastName="Khan" department="Engineering" />
      <EmployeeCard firstName="Ravi" lastName="Mehta" department="Sales" />
    </div>
  );
}
```

Same component, different inputs, different output. That's reuse.

It's cleaner to **destructure** the props right in the parameter list (just like Day 1 object destructuring):

```
function EmployeeCard({ firstName, lastName, department }: EmployeeCardProps) {
  return (
    <div className="card">
      <h2>{firstName} {lastName}</h2>
      <p>{department}</p>
    </div>
  );
}
```

The most important rule about props: props are READ-ONLY. A component may never change its own props. Treat them like `readonly` fields in C# — inputs you look at, never edit.

```
function EmployeeCard({ department }: EmployeeCardProps) {
  // ❌ Never do this. A component must not change its own props.
  // department = "Marketing";
  return <p>{department}</p>;
}
```

Why so strict? Because props always flow *down* from parent to child. If a child could rewrite its props, data could change in two directions and you'd never be sure who changed what. Read-only props keep the flow simple and predictable. (When you need data that *changes over time*, that's **state** — and that's tomorrow, Day 4.)

The `children` prop and composition

Sometimes a component should wrap *whatever you put inside it* — like a box that doesn't care what's in it. React gives every component a special prop called `children`: it holds whatever you place between the opening and closing tags.

```
import type { ReactNode } from "react";

interface CardProps {
  children: ReactNode; // ReactNode = "any renderable content": text, elements, lists ...
}

function Card({ children }: CardProps) {
  return <div className="card">{children}</div>;
}
```

Now `Card` is a reusable frame. You pass content *inside* it:

```
function Example() {
  return (
    <Card>
      <h2>Engineering</h2>
      <p>12 people</p>
    </Card>
  );
}
```

Everything between `<Card>` and `</Card>` arrives as `children`, and `Card` renders it inside its styled `div`.

This nesting of small components to build bigger ones is called **composition** — building by combining. It's the main way you structure a React app. Instead of one giant component, you compose many small ones. Here's a `Dashboard` composing smaller cards:

```
interface DepartmentCardProps {
  name: string;
  headcount: number;
}

function DepartmentCard({ name, headcount }: DepartmentCardProps) {
  return (
    <div className="card">
      <h2>{name}</h2>
      <p>{headcount} people</p>
    </div>
  );
}

function Dashboard() {
  return (
    <div>
      <h1>Company Dashboard</h1>
      <DepartmentCard name="Engineering" headcount={12} />
      <DepartmentCard name="Sales" headcount={8} />
    </div>
  );
}
```

Notice `headcount={12}` uses `{ }` because `12` is a number (a JavaScript value), while `name="Engineering"` uses quotes because it's a plain string. Small detail, easy to trip on.

How rendering works — data flows down, in one direction

Let's connect the dots on what actually happens when React shows your UI.

1. You write **JSX**.
2. JSX becomes **React elements** — tiny plain JavaScript objects that describe what you want (like `{ type: "h2", props: { children: "Aisha" } }`). These are just a *description*, not the real screen yet.
3. React reads that description and updates the **real DOM** (the actual page the browser shows) to match.

So the pipeline is: **JSX → React elements (a description) → DOM (the real screen).** You only ever write the first part. React handles the rest.

Data moves in **one direction: down.** A parent passes props to its children. Children pass props to *their* children. Data never flows back up on its own. This is called **one-way data flow**, and it's a deliberate feature. Because data only goes down, when something looks wrong you always know where to look: the parent that supplied it. In C# terms, it's like arguments flowing into nested method calls — the callee can read them but can't reach back up and reassign the caller's variables.

When does a component re-render (run again)? Keep it simple for today:

- React calls your component function to get its JSX.
- If the **props coming in are different** next time, React calls that function again — that's a **re-render** — and updates only the parts of the screen that actually changed.

What makes props change in a living app? Usually **state** — data that changes over time in response to clicks, typing, or data loading. State is the engine that drives re-renders, and it's the whole story of **Day 4.** For today, just hold this: *same inputs → same UI; different inputs → React re-renders and syncs the screen.*

This is the big difference from Knockout again. In Knockout, you subscribe each observable to each binding, and only those exact bindings update. In React, you don't subscribe anything by hand — React re-runs the component and figures out the differences for you.

Conditional rendering — showing UI only sometimes

Because `{ }` holds JavaScript expressions, showing things conditionally is just normal JS. Three common patterns:

1. `&&` — show something, or show nothing. Read `a && b` as "if `a` is true, produce `b`; otherwise produce nothing."

```
interface StatusProps {
  isActive: boolean;
}

function ActiveBadge({ isActive }: StatusProps) {
  return <div>{isActive && <span className="badge">Active</span>}</div>;
}
```

One caution with `&&`: only use it with a real `true` / `false`. Avoid `{count && ...}` where `count` could be `0`, because `0` is falsy and React would print the literal `0` on screen. Use a proper condition like `{count > 0 && ...}`.

2. Ternary `? :` — choose between two options. Read `cond ? a : b` as "if `cond`, `a`, else `b`."

```
function StatusLabel({ isActive }: StatusProps) {
  return <span>{isActive ? "Working" : "On leave"}</span>;
}
```

3. Early return — bail out before the main UI. Great for "no data" cases.

```
interface EmployeeCardProps {
  employee: Employee | null;
}

function EmployeeCard({ employee }: EmployeeCardProps) {
  if (!employee) {
    return <p>No employee selected.</p>;
  }
  // From here down, TypeScript knows employee is not null (Day 2 narrowing!)
  return <h2>{employee.firstName} {employee.lastName}</h2>;
}
```

That early return also shows a Day 2 idea paying off: after the `if (!employee) return`, TypeScript **narrows** `employee` to a real `Employee`, so `employee.firstName` is safe with no red squiggle.

Lists and keys — turning an array into UI

Real screens show *lists* — a list of employees, of departments. You turn an array into elements with `.map()`, the same array method from Day 1. Each item becomes a piece of JSX.

```
interface EmployeeListProps {
  employees: Employee[];
}

function EmployeeList({ employees }: EmployeeListProps) {
  return (
    <ul>
      {employees.map((emp) => (
        <li key={emp.id}>
          {emp.firstName} {emp.lastName} - {emp.department}
        </li>
      ))}
    </ul>
  );
}
```

See that `key={emp.id}`? **Every element in a mapped list needs a `key` — a value that is unique among its siblings.** This is not optional; React will warn you if it's missing.

Why keys matter. A key is an *identity tag*. It tells React "this specific `<li>` is the *same* item as before." When the list changes — an employee is added, removed, or reordered — React uses keys to match old

items to new ones. With good keys, React moves and reuses the exact rows that changed and leaves the rest alone. Without them (or with bad ones), React can get confused and rebuild or mismatch rows, which is slow and can scramble things like input focus.

Use a **stable, unique id** for the key — `emp.id` is perfect. **Avoid using the array index** as the key when the list can be reordered, filtered, or edited, because the index of an item changes when the list changes, so it no longer identifies the same item.

```
// ❌ Avoid this when the list can change order or size:
// {employees.map((emp, index) => <li key={index}> ... </li>)}
```

The Virtual DOM and reconciliation, in plain words

You'll hear these two fancy terms a lot. Here they are in plain English.

The **Virtual DOM** is React's lightweight in-memory copy of your UI — those plain JavaScript "React element" objects from earlier. It's cheap to build because it's just objects, not real screen elements.

When your data changes, React builds a **new** Virtual DOM (the new description of how things should look) and compares it against the previous one. This compare-and-find-the-differences step is called **reconciliation.** React works out the smallest set of real changes needed, then touches the actual DOM only in those spots.

Why bother? Because touching the real DOM directly is slow, and doing it a lot by hand (the Knockout way, wire by wire) is easy to get wrong. React lets you re-describe the *whole* UI whenever data changes — simple to think about — while quietly making only the tiny real updates that are actually needed. You get simple code *and* good performance.

Project structure — one component per file

React doesn't force a folder layout, but there's a strong, simple convention: **one component per file**, named after the component. `EmployeeCard.tsx` holds `EmployeeCard`, `Dashboard.tsx` holds `Dashboard`. This is just like the C# habit of one class per file.

You share components between files with `export` / `import` — the ES module syntax from Day 1:

```tsx
// EmployeeCard.tsx
import type { Employee } from "./employee";

interface EmployeeCardProps {
  employee: Employee;
}

export function EmployeeCard({ employee }: EmployeeCardProps) {
  return (
    <div className="card">
      <h2>{employee.firstName} {employee.lastName}</h2>
      <p>{employee.department}</p>
    </div>
  );
}
```

```tsx
// Dashboard.tsx
import { EmployeeCard } from "./EmployeeCard";
import type { Employee } from "./employee";

const sample: Employee = {
  id: 1,
  firstName: "Aisha",
  lastName: "Khan",
  department: "Engineering",
  salary: 90000,
  isActive: true,
  joinDate: "2023-04-01",
};

export function Dashboard() {
  return (
    <div>
      <h1>Company Dashboard</h1>
      <EmployeeCard employee={sample} />
    </div>
  );
}
```

A typical small project looks like this:

```
src/
  main.tsx          ← the entry point: mounts React onto the page
  App.tsx           ← the top component
  employee.ts       ← the Employee type (and any helpers)
  components/
    Dashboard.tsx
    DepartmentCard.tsx
    EmployeeCard.tsx
```

And the SPA "single page" idea becomes concrete in `main.tsx` — this is the *one* place React grabs a spot in the HTML page and takes over from there:

```
// main.tsx
import { createRoot } from "react-dom/client";
import { Dashboard } from "./components/Dashboard";

// index.html has one <div id="root"></div>. React fills it and runs the whole app inside
it.
createRoot(document.getElementById("root")!).render(<Dashboard />);
```

That single `<div id="root">` is the "single page." Everything you build lives inside it, and React swaps its contents as your data changes — no page reloads.

One-line summary

React = you write small components (functions that return JSX) and describe the UI for your current data; props flow down, read-only; React re-renders and quietly updates only what changed — you describe the *what*, React handles the *how*.

3 Common Mistakes to Avoid

These are the classic slip-ups when you first come to React from KnockoutJS and .NET. Each one is short: the ❌ wrong way, the ✅ right way, and a one-line "why." Read once, and you will dodge 90% of beginner React bugs.

> Reminder: any ❌ code below is shown as a **comment** on purpose (so nothing broken runs). The ✅ code is the real, working version.

---

1. `class=` and `for=` don't work — use `className` and `htmlFor`

```
// ❌ Wrong — HTML habit. React quietly ignores class= and for=.
// <div class="card">
//   <label for="dept">Department</label>
// </div>

// ✅ Right — className and htmlFor
function Field() {
  return (
    <div className="card">
      <label htmlFor="dept">Department</label>
      <input id="dept" />
    </div>
  );
}
```

Why: JSX is JavaScript, not HTML. In JavaScript, `class` and `for` are already reserved words (a class, and a `for` loop). So React renames them to `className` and `htmlFor`. In KnockoutJS your views were plain HTML, so `class="..."` was fine there — here it is different because you are inside JS.

---

2. A component must return ONE root — wrap extras in a fragment `<> ... </>`

```
// ❌ Wrong — two elements side by side, no single parent. Won't compile.
// function Header() {
//   return (
//     <h1>Employees</h1>
//     <p>All staff</p>
//   );
// }

// ✅ Right — a fragment groups them without adding an extra <div>
function Header() {
  return (
    <>
      <h1>Employees</h1>
      <p>All staff</p>
    </>
  );
}
```

Why: A component returns one thing, like a C# method returns one object. A fragment `<> … </>` is an invisible wrapper — it groups children but adds nothing to the actual page.

---

3. Never mutate (change) props — data flows DOWN, one way

```
interface EmployeeCardProps {
  firstName: string;
  salary: number;
}

// ❌ Wrong — props are read-only. Never reassign them.
// function EmployeeCard(props: EmployeeCardProps) {
//   props.salary = props.salary + 1000; // ❌ don't touch the prop
//   return <p>{props.firstName}</p>;
// }

// ✅ Right — read the prop, and make a NEW local value if you need a change
function EmployeeCard(props: EmployeeCardProps) {
  const raisedSalary = props.salary + 1000; // prop untouched
  return (
    <p>
      {props.firstName}: {raisedSalary}
    </p>
  );
}
```

Why: Props are a gift from the parent — you read them, you don't rewrite them. React data flows **one way, downward** (parent → child). This is the big shift from KnockoutJS: there, `data-bind` was often **two-way** and the child could push changes back up automatically. In React, the child never edits props; if something must change, the parent owns it (and state, coming Day 4, handles the change). Tip: you can even force this in TypeScript with `readonly` fields on the interface.

---

4. Every list item needs a stable `key` — and don't use the array index

```tsx
interface Employee {
  id: number;
  firstName: string;
}

// ❌ Wrong — no key. React warns, and list updates get buggy.
// {employees.map((e) => <li>{e.firstName}</li>)}

// ❌ Risky — index as key breaks when the list reorders, filters, or an item is removed.
// {employees.map((e, index) => <li key={index}>{e.firstName}</li>)}

// ✅ Right — use a stable, unique id
function EmployeeList({ employees }: { employees: Employee[] }) {
  return (
    <ul>
      {employees.map((e) => (
        <li key={e.id}>{e.firstName}</li>
      ))}
    </ul>
  );
}
```

Why: A `key` is React's way to tell rows apart between renders — think of it like a primary key on a DB table. If you use the index, the "key" changes whenever the list reorders or an item is removed, so React can attach the wrong data to the wrong row. Use the record's real `id`.

---

5. No `if` statement directly inside JSX — use `? :`, `&&`, or an early return

```
interface Employee {
  firstName: string;
  isActive: boolean;
}

// ❌ Wrong — a raw if-statement can't live inside JSX braces.
// function Status({ employee }: { employee: Employee }) {
//   return <span>{ if (employee.isActive) { "Active" } }</span>; // ❌ syntax error
// }

// ✅ Right — ternary and && are EXPRESSIONS (they return a value)
function Status({ employee }: { employee: Employee }) {
  return (
    <span>
      {employee.isActive ? "Active" : "Inactive"}
      {employee.isActive && <strong> ✓</strong>}
    </span>
  );
}

// ✅ Also fine — early return when the whole component has two branches
function StatusBadge({ employee }: { employee: Employee }) {
  if (!employee.isActive) {
    return <span>Inactive</span>;
  }
  return <span>Active</span>;
}
```

Why: The `{ }` inside JSX holds an **expression** — something that produces a value. `if` is a **statement** — it does something but hands back no value, so JSX can't display it. `? :` and `&&` both produce a value, so they fit. (An `if` is fine up top, outside the returned JSX, like in the early-return example.)

---

6. Put an EXPRESSION in `{ }`, not a statement (no `for` loops, no `const`)

```
interface Employee {
  id: number;
  firstName: string;
}

// ❌ Wrong — a for-loop and a const are statements, not allowed inside { }.
// <ul>
//   { for (const e of employees) { <li>{e.firstName}</li> } }  // ❌
// </ul>

// ✅ Right — .map() is an expression: it returns an array of elements
function Names({ employees }: { employees: Employee[] }) {
  return (
    <ul>
      {employees.map((e) => (
        <li key={e.id}>{e.firstName}</li>
      ))}
    </ul>
  );
}
```

Why: Same rule as #5, wider view. Inside `{ }` React wants a **value** it can render. A loop or a variable declaration gives no value. `.map()` returns an **array** of elements — and React happily renders arrays. Do your loops and variables *above* the `return`, then drop the result into `{ }`.

---

7. Component names must start with a Capital letter (PascalCase)

```
// ❌ Wrong — lowercase. React thinks <employeeCard /> is an unknown HTML tag.
// function employeeCard() {
//   return <div>An employee</div>;
// }
// ... used later as <employeeCard />  // ❌ treated as HTML, renders nothing useful

// ✅ Right — PascalCase, so React knows it's YOUR component
function EmployeeCard() {
  return <div>An employee</div>;
}
// ... used later as <EmployeeCard />
```

Why: React decides by the first letter. **lowercase** = a built-in HTML tag (`div` , `span` , `input`). **Uppercase** = your own component. It's the same PascalCase habit you already use for C# class names — but here

React genuinely depends on it to work.

---

8. Render a component as `<Component />`, don't call it like `Component()`

```
function EmployeeCard() {
  return <div>An employee</div>;
}

function Dashboard() {
  return (
    <div>
      {/* ❌ Wrong — calling it like a plain function */}
      {/* {EmployeeCard()} */}

      {/* ✅ Right — render it as a tag so React manages it properly */}
      <EmployeeCard />
    </div>
  );
}
```

Why: `<EmployeeCard />` tells React "this is a real component — track it, give it its own identity." Calling `EmployeeCard()` just runs the function once and pastes its output inline, so React can't manage it as a component (this breaks re-rendering and hooks later). Always use the tag form.

---

9. Match your export style to your import style (default vs named)

```
// --- EmployeeCard.tsx ---
// Option A — DEFAULT export
export default function EmployeeCard() {
  return <div>Card</div>;
}

// Option B — NAMED export (pick ONE style per file)
// export function EmployeeCard() { return <div>Card</div>; }
```

```
// --- Dashboard.tsx ---
// ❌ Wrong – import style doesn't match the export style:
// import { EmployeeCard } from "./EmployeeCard"; // braces, but file used DEFAULT export
// import EmployeeCard from "./EmployeeCard";      // no braces, but file used NAMED export

// ✅ Right – match them:
// import EmployeeCard from "./EmployeeCard";      // for `export default`
// import { EmployeeCard } from "./EmployeeCard";  // for named `export function`
```

Why: Default export → import with **no braces** (`import X from "..."`). **Named export** → import **with braces** (`import { X } from "..."`). Mismatch them and you get `undefined` or "has no exported member" errors. It's a bit like a C# `using` plus the exact type name — but in React the **braces** are the switch. Also: if you forget to `export` at all, the other file simply can't see the component.

---

10. Stop thinking in the DOM — think in state/props (the Knockout/jQuery habit)

```
interface Employee {
  firstName: string;
  isActive: boolean;
}

// ❌ Old KnockoutJS/jQuery habit – grab the element and poke it by hand:
// const el = document.getElementById("status");
// el!.innerText = employee.isActive ? "Active" : "Inactive";

// ✅ React way – describe the UI from the data; React updates the DOM for you
function EmployeeStatus({ employee }: { employee: Employee }) {
  return <span>{employee.isActive ? "Active" : "Inactive"}</span>;
}
```

Why: In KnockoutJS and jQuery you find a node and change it yourself. In React you almost never touch the DOM. You describe **what the screen should look like for the current data**, and React works out the DOM edits. The mental model is `UI = f(data)` ("UI is a function of the data") — not "find the element and edit it." Data flows down (props today, state tomorrow), and the screen simply follows.

Quick recap

❌ Don't	✅ Do
`class=` , `for=`	`className=` , `htmlFor=`

❌ Don't	✅ Do
Return two elements loose	Wrap in a fragment `<> … </>`
Change a prop	Treat props as read-only; make a new value
No `key`, or `key={index}`	`key={item.id}` (stable, unique)
`if` inside JSX braces	`? :`, `&&`, or early `return`
`for` loop / `const` in `{ }`	`.map()` (an expression) above or inside `{ }`
`employeeCard` (lowercase)	`EmployeeCard` (PascalCase)
`{EmployeeCard()}`	`<EmployeeCard />`
Default vs named import mismatch	Match export style ↔ import style
"Find the node and edit it"	Describe UI from data; let React update the DOM

4 Concept Notes — Recall & Interview (copy by pen)

> Quick bullet notes for concept recall + classic React interview questions — copy these by pen and revise fast.

What React Is (the big picture)

React

Definition:

- React is a JavaScript library for building user interfaces out of small, reusable pieces called components.

✅ **Advantages:**

- Component-based — build UI like Lego blocks and reuse them anywhere.
- Declarative — you describe *what* the UI should look like; React works out *how* to update the screen.

❌ **Disadvantages / watch-outs:**

- It's a library, not a full framework — routing, data-fetching, etc. are separate add-ons you choose.

💡 **Interview tip:**

- "Library or framework?" → say *library* (it only handles the UI layer).

📝 **Example:**

- `<EmployeeCard />` — one reusable UI piece you can drop in many times.
- C#/Knockout: like a Razor partial view or a Knockout component — a chunk of UI you reuse.

Declarative vs Imperative

Definition:

- Declarative = you state the end result you want; imperative = you write step-by-step DOM instructions by hand.

✅ **Advantages:**

- Less manual DOM code, so fewer bugs.

📝 **Example:**

- `return <p>{count}</p>;` — you declare the output; React updates the real DOM for you.
- C#/Knockout: Knockout `data-bind` is declarative too; raw jQuery `$el.text( ... )` is imperative.

SPA (Single Page Application)

Definition:

- An app that loads one HTML page once, then swaps content with JavaScript instead of asking the server for a whole new page on each click.

✅ **Advantages:**

- Fast, smooth navigation — no full page reload or flicker.

❌ **Disadvantages / watch-outs:**

- Heavier first load; SEO and initial load need extra care.
- C#/.NET: unlike classic ASP.NET MVC, where each click returns a fresh server-rendered page.

JSX & Components

JSX

Definition:

- JSX is HTML-like syntax you write inside JavaScript — it is not a string and not real HTML.

✅ **Advantages:**

- UI and logic live together; drop `{ }` to embed any JS value right in the markup.

❌ **Disadvantages / watch-outs:**

- It compiles to function calls — `<h1>Hi</h1>` becomes `React.createElement("h1", null, "Hi")` (or `_jsx( ... )`), so JSX is really just JavaScript.
- A component must return one parent element (or a Fragment).

💡 **Interview tip:**

- "Is JSX HTML?" → No. It's syntax sugar that compiles to `createElement` calls returning plain JS objects (React elements).

📝 **Example:**

- `const el = <h1>Hi</h1>;` — looks like HTML, is actually a JS object.

JSX rules vs HTML

Definition:

- JSX looks like HTML but follows JS naming — attributes are camelCase and a few names differ.

❌ **Disadvantages / watch-outs:**

- Use `className` not `class`, `htmlFor` not `for`, `onClick` not `onclick`.

📝 **Example:**

- `<div className="card" />` — self-closing tags need the trailing slash.

className vs class

Definition:

- In JSX you set CSS classes with `className`, because `class` is a reserved keyword in JavaScript.

💡 **Interview tip:**

- Classic gotcha: "Why className, not class?" → `class` is reserved in JS.

📝 **Example:**

- `<span className="badge">Active</span>` — renders as `class="badge"` in the real DOM.

Function components

Definition:

- A function component is just a JavaScript function that returns JSX (the UI).

✅ **Advantages:**

- Plain functions — simple, easy to test and reuse; the modern default.

❌ **Disadvantages / watch-outs:**

- Component names MUST start with a Capital letter, or React treats it as a plain HTML tag.
- (Class components exist but are legacy — you'll almost never write one now.)

📝 **Example:**

- `function EmployeeCard(props: { name: string }) { return <div>{props.name}</div>; }` — a component is a function returning JSX.
- C#/.NET: like a small method/partial that returns a piece of markup.

Typing props with an interface (TS)

Definition:

- In TypeScript you describe the shape of a component's props with an `interface`, so you get autocomplete and an error if a prop is missing or the wrong type.

✅ **Advantages:**

- Compile-time safety — a wrong prop type is caught before the app runs.

📝 **Example:**

- `function EmployeeCard({ name }: { name: string }) { return <div>{name}</div>; }` — props are typed.
- C#/.NET: like the parameter types on a method signature — the compiler checks every caller.

Fragments

Definition:

- A Fragment lets a component return several elements without adding an extra wrapper `<div>` to the DOM.

✅ **Advantages:**

- Cleaner HTML — no useless wrapper nodes.

📝 **Example:**

- `<>{a}{b}</>` — shorthand Fragment groups siblings and renders nothing extra.

Props, Composition & Data Flow

Props

Definition:

- Props (properties) are the inputs a parent passes down to a child component.

✅ **Advantages:**

- Make components reusable — same component, different data each time.

❌ **Disadvantages / watch-outs:**

- Props are read-only — a child must never change its own props.

💡 **Interview tip:**

- "Can a child change its props?" → No, props are read-only; the child only reads them.

📝 **Example:**

- `<EmployeeCard name="Asha" salary={50000} />` — strings in quotes, other JS values in `{ }`.
- C#/.NET: like passing arguments into a method or setting properties on an object.

One-way data flow

Definition:

- Data flows in one direction only: from parent down to child through props (top → down).

✅ **Advantages:**

- Predictable — you always know where data came from, so bugs are easier to trace.

❌ **Disadvantages / watch-outs:**

- To send info back up, the parent passes a function down (that's Day 4 — events/state).

💡 **Interview tip:**

- Classic: "What is one-way data flow?" → data goes parent → child; children can't push data upward directly.
- C#/Knockout: Knockout two-way binding syncs both ways; React is deliberately one-way for predictability.

Props immutability / pure rendering

Definition:

- A component should be pure: given the same props, it always returns the same UI and never modifies its inputs.

✅ **Advantages:**

- Same input → same output makes rendering predictable and easy to reason about.

❌ **Disadvantages / watch-outs:**

- Never mutate props (or outside data) during render — treat props as read-only.

💡 **Interview tip:**

- "Why are props immutable?" → keeps rendering predictable and enables React's one-way flow and optimizations.

📝 **Example:**

- `// ❌ props.salary = 0` — never reassign props; read them only.
- C#/.NET: like a pure method with no side effects — same args, same result.

Component composition

Definition:

- Composition means building bigger UI by nesting small components inside each other, instead of one giant component.

✅ **Advantages:**

- Reuse and clean structure — a `Dashboard` uses `DepartmentCard`, which uses `EmployeeCard`.

📝 **Example:**

- `<Dashboard><DepartmentCard /></Dashboard>` — components nested to form the page.
- C#/Knockout: like a page built from many partial views / nested Knockout components.

children prop

Definition:

- `children` is a special prop holding whatever JSX you put between a component's opening and closing tags.

✅ **Advantages:**

- Lets you build flexible wrappers (cards, layouts) that don't care what's inside them.

📝 **Example:**

- `function Card(props: { children: React.ReactNode }) { return <div className="card"> {props.children}</div>; }` — inner JSX arrives via `children`.
- C#/Knockout: like Razor `@RenderBody()` / a layout placeholder, or Knockout's `template` content.

Controlled-by-parent idea

Definition:

- A child's look and data are decided by the props its parent gives it — the parent is in charge.

✅ **Advantages:**

- The same child behaves differently just by changing what the parent passes.

📝 **Example:**

- `<EmployeeCard isActive={false} />` — parent decides; child just displays what it's told.

Rendering, Virtual DOM & Keys

Rendering

Definition:

- Rendering means React calls your component function to get its JSX, then shows that as UI on the screen.

📝 **Example:**

- `ReactDOM.createRoot(el).render(<App />)` — the first render puts your app on the page.

Re-rendering

Definition:

- Re-rendering is React calling a component again to make fresh UI when its inputs change (mainly when props or state change).

✅ **Advantages:**

- UI stays in sync with data automatically — you never touch the DOM by hand.

❌ **Disadvantages / watch-outs:**

- Re-render ≠ full page reload; it just re-runs the component function (state comes tomorrow, Day 4).
- C#/Knockout: like a Knockout observable updating its bound DOM — but React re-runs the whole component function.

Virtual DOM

Definition:

- The virtual DOM is a lightweight copy of the UI kept in memory as plain JS objects; React updates it first, not the real DOM directly.

✅ **Advantages:**

- Updating JS objects is cheap; React then applies only the needed real-DOM changes.

💡 **Interview tip:**

- Very classic: "What is the virtual DOM and why?" → an in-memory UI tree; React diffs old vs new and updates only what changed, because real DOM updates are slow.

- C#/.NET: think of it as an in-memory model React syncs to the "view" (real DOM) for you.

Reconciliation

Definition:

- Reconciliation is React comparing (diffing) the new virtual DOM against the old one to find the smallest set of real-DOM changes to make.

✅ **Advantages:**

- Only the changed parts of the page are touched — fast and efficient.

💡 **Interview tip:**

- "How does React update the DOM efficiently?" → it diffs virtual DOM trees (reconciliation) and patches only the differences.

📝 **Example:**

- change one `<td>` in a table → only that cell updates, not the whole table.

Keys

Definition:

- A `key` is a unique id you give each item when rendering a list, so React can tell the items apart between renders.

✅ **Advantages:**

- Correct, stable updates — React reuses the right DOM nodes instead of rebuilding the list.

❌ **Disadvantages / watch-outs:**

- Don't use the array index as key if the list can reorder/add/remove — it causes wrong UI and subtle bugs.
- Keys must be unique among siblings; use a stable id like `employee.id`.

💡 **Interview tip:**

- Super classic: "Why are keys needed?" → they help React match old vs new list items during reconciliation, avoiding wasteful or wrong re-renders.

📝 **Example:**

- `list.map(e => <EmployeeCard key={e.id} name={e.firstName} />)` — `key` uses the stable id.

Lists with map

Definition:

- You render a list by calling `.map()` on an array and returning one component per item.

📝 **Example:**

- `employees.map(e => <EmployeeCard key={e.id} name={e.firstName} />)` — array → array of UI elements.
- C#/.NET: like a `foreach` / LINQ `Select` that turns a collection into markup.

Conditional rendering

Definition:

- Conditional rendering means showing different JSX based on a condition, using `&&`, a ternary, or an early `return`.

✅ **Advantages:**

- Easily show/hide parts of the UI, like an "Inactive" badge.

❌ **Disadvantages / watch-outs:**

- `{count && <X/>}` prints `0` when count is 0 — use a real boolean or a ternary instead.

📝 **Example:**

- `{isActive ? <span>Active</span> : <span>Left</span>}` — a ternary picks which UI to show.
- C#/Knockout: like Knockout's `if` / `visible` binding.